In [1]:
```
import numpy
import matplotlib.pyplot as plt
import sklearn as sk
```

Preprocess

In [2]:
```
import os
import pandas as pd

data_dir = "./data/SRP192714"
data_name = "SRP192714_Symbols.tsv"
metadata_name = "metadata_SRP192714_merged.tsv"
data_path = os.path.join(data_dir, data_name)
metadata_path = os.path.join(data_dir, metadata_name)

data = pd.read_csv(data_path, sep="\t", header=0)
with open(metadata_path, 'r', encoding='utf-8', errors='replace') as fh:
    metadata = pd.read_csv(fh, sep='\t', header=0)
```

In [3]:
```
import numpy as np
numeric = data.select_dtypes(include=[np.number])
gene_var = numeric.var(axis=1)
data['gene_variance'] = gene_var
data_sorted = data.sort_values(by='gene_variance', ascending=False)
data_top5000 = data_sorted.head(5000).dropna(subset=['Symbol'])
metadata_clean = metadata[['refinebio_accession_code', 'Exposure']].dropna()
metadata_clean = metadata_clean.rename(columns={'refinebio_accession_code':
metadata_clean['patient_id'] = metadata_clean['sample_id'].str.extract(r'(SR
metadata_clean = metadata_clean.dropna(subset=['patient_id'])
```

In [4]:
```
data_top5000
```

Out[4]:

	Symbol	SRR8907879	SRR8907880	SRR8907881	SRR8907882	SRR89078
24102	XIST	3.427381	4.144367	3.255694	2.847241	3.3463
6527	JCHAIN	69.905350	70.659115	67.071661	65.554728	67.4103
39232	RN7SL1	78.682248	78.186574	80.118708	79.648319	79.6483
7499	IFI44L	93.356831	96.637429	91.828849	92.607267	95.3126
4944	IFIT3	71.775701	73.690123	76.826814	72.160454	71.7757
...	...	...	...	...	...	
11281	ELAPOR2	3.916609	2.900281	4.883377	3.696960	4.2499
20652	IGHV1-46	1.103100	0.196625	1.432086	1.112272	1.1095
21282	RPL17-C18orf32	2.629247	0.196625	2.726308	0.199846	2.5710
10931	EIF5A2	3.580384	4.723720	5.615663	3.404488	5.5436
16678	ZMYM1	5.792604	5.896217	7.699045	6.500447	6.7738

4847 rows × 1023 columns

In [5]:
```
print(f"Samples with exposure labels: {len(metadata_clean)}")
print(f"Unique patients: {metadata_clean['patient_id'].nunique()}")
print(f"Exposure distribution:\n{metadata_clean['Exposure'].value_counts()}"
```

```
Samples with exposure labels: 1021
Unique patients: 1021
Exposure distribution:
Exposure
naive    553
DENV     468
Name: count, dtype: int64
```

In [6]:
```
# Prepare expression matrix (samples x genes)
expr_samples = data_top5000.drop(columns=['Symbol', 'gene_variance']).columr
common_samples = list(set(expr_samples) & set(metadata_clean['sample_id']))

# Transpose expression data to align with metadata
expr_matrix = data_top5000.drop(columns=['Symbol', 'gene_variance'])[common_
metadata_aligned = metadata_clean[metadata_clean['sample_id'].isin(common_sa
metadata_aligned = metadata_aligned.set_index('sample_id').loc[expr_matrix.i

print(f"Final matrix shape: {expr_matrix.shape}")
print(f"Features (genes): {expr_matrix.shape[1]}")
print(f"Samples: {expr_matrix.shape[0]}")
```

```
Final matrix shape: (1021, 4847)
Features (genes): 4847
Samples: 1021
```

Patient-Level Data Splitting

In [7]:
```
from sklearn.model_selection import GroupShuffleSplit

# Get patient-level labels
patient_labels = metadata_aligned.groupby('patient_id')['Exposure'].first()

X_all = expr_matrix
y_all = metadata_aligned['Exposure']
groups_all = metadata_aligned['patient_id']

gss = GroupShuffleSplit(n_splits=1, test_size=0.3, random_state=42)
train_idx, test_idx = next(gss.split(X_all, y_all, groups=groups_all))

# train/test splits
X_train = X_all.iloc[train_idx]
X_test = X_all.iloc[test_idx]
y_train = y_all.iloc[train_idx]
y_test = y_all.iloc[test_idx]

# for later
train_patient_ids = groups_all.iloc[train_idx].unique()
test_patient_ids = groups_all.iloc[test_idx].unique()

print(f"Training: {X_train.shape[0]} samples, {len(train_patient_ids)} patie
print(f"Testing: {X_test.shape[0]} samples, {len(test_patient_ids)} patients
print(f"Train labels: {y_train.value_counts().to_dict()}")
print(f"Test labels: {y_test.value_counts().to_dict()}")
```

```
Training: 714 samples, 714 patients
Testing: 307 samples, 307 patients
Train labels: {'naive': 383, 'DENV': 331}
Test labels: {'naive': 170, 'DENV': 137}
```

Binary Classification: DENV vs Naive

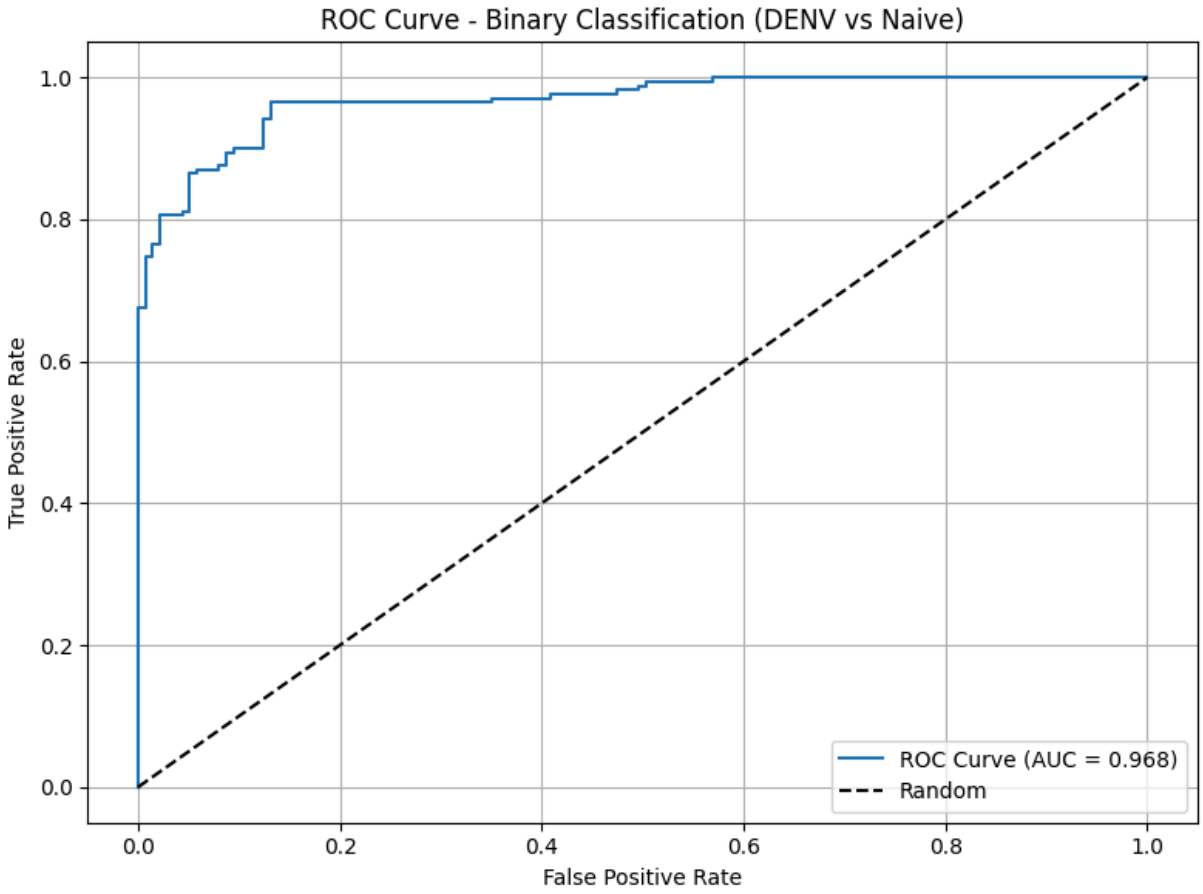
In [8]:
```
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.pipeline import Pipeline
from sklearn.metrics import roc_auc_score, roc_curve, classification_report
import os

os.makedirs('results', exist_ok=True)
os.makedirs('plots', exist_ok=True)

# Convert string labels to binary
label_encoder = LabelEncoder()
y_train_binary = label_encoder.fit_transform(y_train)
y_test_binary = label_encoder.transform(y_test)

print(f"Label mapping: {dict(zip(label_encoder.classes_, [0, 1]))}")

# SVM pipeline with scaling
```

```
svm_binary = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', SVC(kernel='rbf', probability=True, random_state=42))
])

svm_binary.fit(X_train, y_train_binary)

y_pred_proba = svm_binary.predict_proba(X_test)[:, 1]
y_pred = svm_binary.predict(X_test)

# Calculate ROC AUC
auc_score = roc_auc_score(y_test_binary, y_pred_proba)
print(f"Binary Classification AUC: {auc_score:.3f}")

print("\nClassification Report:")
print(classification_report(y_test_binary, y_pred, target_names=label_encode
```

```
Label mapping: {'DENV': 0, 'naive': 1}
Binary Classification AUC: 0.968

Classification Report:
              precision    recall  f1-score   support

        DENV       0.87      0.91      0.89       137
       naive       0.93      0.89      0.91       170

    accuracy                           0.90       307
   macro avg       0.90      0.90      0.90       307
weighted avg       0.90      0.90      0.90       307
```

In [9]:

```
# ROC Curve for Binary Classification
fpr, tpr, _ = roc_curve(y_test_binary, y_pred_proba)

plt.figure(figsize=(8, 6))
plt.plot(fpr, tpr, label=f'ROC Curve (AUC = {auc_score:.3f})')
plt.plot([0, 1], [0, 1], 'k--', label='Random')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curve - Binary Classification (DENV vs Naive)')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig('results/binary_roc_curve.png', dpi=300, bbox_inches='tight')
plt.show()
```



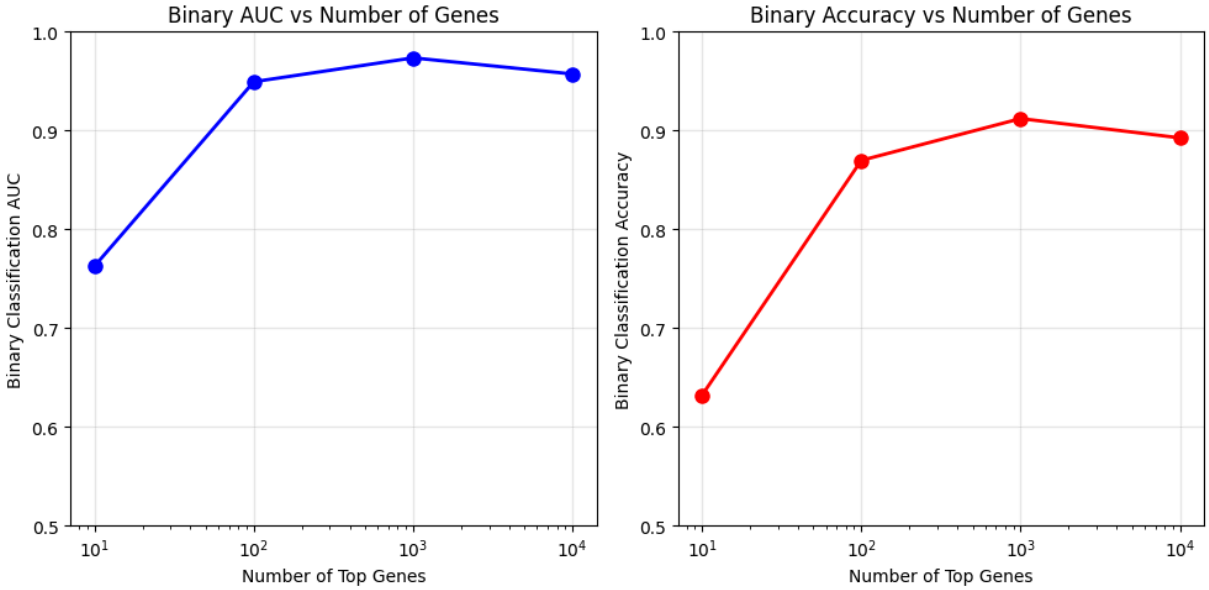
In [10]:
```python
# SVM Analysis with Different Numbers of Top Genes

import numpy as np
import pandas as pd
from sklearn.svm import SVC
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.pipeline import Pipeline
from sklearn.metrics import roc_auc_score, accuracy_score, classification_re
import matplotlib.pyplot as plt

# Gene counts to test
gene_counts = [10, 100, 1000, 10000]
results_summary = {}

# Storage for results
all_results = []

print("=== SVM Analysis with Different Numbers of Top Genes ===\n")

for n_genes in gene_counts:
    print(f"Testing with top {n_genes} genes...")

    # Select top n genes
    if n_genes <= len(data_sorted):
        data_subset = data_sorted.head(n_genes)
    else:
        print(f"Only {len(data_sorted)} genes available, using all")
        data_subset = data_sorted
```

```
        n_genes = len(data_sorted)

    # Create expression matrix for this subset
    expr_matrix_subset = data_subset.drop(columns=['Symbol', 'gene_variance'
    expr_matrix_subset.columns = expr_matrix_subset.columns.astype(str)  # F

    # Use same train/test split
    X_train_subset = expr_matrix_subset.iloc[train_idx]
    X_test_subset = expr_matrix_subset.iloc[test_idx]

    # Binary Classification
    svm_binary_subset = Pipeline([
        ('scaler', StandardScaler()),
        ('svm', SVC(kernel='rbf', probability=True, random_state=42))
    ])

    svm_binary_subset.fit(X_train_subset, y_train_binary)
    y_pred_proba_subset = svm_binary_subset.predict_proba(X_test_subset)[:,
    y_pred_subset = svm_binary_subset.predict(X_test_subset)

    # Calculate metrics
    auc_subset = roc_auc_score(y_test_binary, y_pred_proba_subset)
    accuracy_binary = accuracy_score(y_test_binary, y_pred_subset)

    # Cluster Classification (3 clusters)
    svm_cluster_subset = Pipeline([
        ('scaler', StandardScaler()),
        ('svm', SVC(kernel='rbf', probability=True, random_state=42))
    ])

    # Store results
    result = {
        'n_genes': n_genes,
        'binary_auc': auc_subset,
        'binary_accuracy': accuracy_binary,
        'n_features': X_train_subset.shape[1]
    }
    all_results.append(result)

    print(f"  Binary AUC: {auc_subset:.3f}")
    print(f"  Binary Accuracy: {accuracy_binary:.3f}")
    print(f"  Features used: {X_train_subset.shape[1]}\n")

# Convert results to DataFrame
results_df = pd.DataFrame(all_results)
print("=== Summary Table ===")
display(results_df)
```

```
=== SVM Analysis with Different Numbers of Top Genes ===

Testing with top 10 genes...
  Binary AUC: 0.763
  Binary Accuracy: 0.632
  Features used: 10

Testing with top 100 genes...
  Binary AUC: 0.949
  Binary Accuracy: 0.870
  Features used: 100

Testing with top 1000 genes...
  Binary AUC: 0.974
  Binary Accuracy: 0.912
  Features used: 1000

Testing with top 10000 genes...
  Binary AUC: 0.957
  Binary Accuracy: 0.893
  Features used: 10000

=== Summary Table ===
```

	n_genes	binary_auc	binary_accuracy	n_features
0	10	0.762602	0.631922	10
1	100	0.949334	0.869707	100
2	1000	0.973508	0.912052	1000
3	10000	0.957192	0.892508	10000

In [11]:
```python
# Plot results
fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(10, 5))

# Binary AUC vs Number of Genes
ax1.semilogx(results_df['n_genes'], results_df['binary_auc'], 'bo-', linewic
ax1.set_xlabel('Number of Top Genes')
ax1.set_ylabel('Binary Classification AUC')
ax1.set_title('Binary AUC vs Number of Genes')
ax1.grid(True, alpha=0.3)
ax1.set_ylim([0.5, 1.0])

# Binary Accuracy vs Number of Genes
ax2.semilogx(results_df['n_genes'], results_df['binary_accuracy'], 'ro-', li
ax2.set_xlabel('Number of Top Genes')
ax2.set_ylabel('Binary Classification Accuracy')
ax2.set_title('Binary Accuracy vs Number of Genes')
ax2.grid(True, alpha=0.3)
ax2.set_ylim([0.5, 1.0])

plt.tight_layout()
plt.savefig('results/gene_count_performance.png', dpi=300, bbox_inches='tigh
plt.show()
```



As we can see, the AUC increases until about 1000 genes. Then, it begins to decrease. This is likely because the SVM is learning the noise associated with housekeeping genes instead of the data that actually matters. This is why it was important to select the top 5000 most variable genes in the first place, I imagine.

Clustering for Multi-class Prediction

In [12]:
```
from sklearn.cluster import AgglomerativeClustering
from sklearn.decomposition import PCA

# Use PCA for dimensionality reduction before clustering
pca = PCA(n_components=50, random_state=42)
X_pca = pca.fit_transform(StandardScaler().fit_transform(X_all))

# Agglomerative clustering (hierarchical from assignment 3
agg_clust_2 = AgglomerativeClustering(n_clusters=2)
agg_clust_3 = AgglomerativeClustering(n_clusters=3)
agg_clust_30 = AgglomerativeClustering(n_clusters=30)
cluster_labels_2 = agg_clust_2.fit_predict(X_pca)
cluster_labels_3 = agg_clust_3.fit_predict(X_pca)

# Add cluster labels to metadata
metadata_with_clusters = metadata_aligned.copy()
metadata_with_clusters['cluster_2'] = cluster_labels_2
metadata_with_clusters['cluster_3'] = cluster_labels_3

print(f"Cluster distribution: {np.bincount(cluster_labels_2)}")
print(f"Cluster distribution: {np.bincount(cluster_labels_3)}")
print(f"Clusters vs Exposure:")
cluster_exposure_table2 = pd.crosstab(metadata_with_clusters['cluster_2'], m
cluster_exposure_table3 = pd.crosstab(metadata_with_clusters['cluster_3'], m
print(cluster_exposure_table2)
print(cluster_exposure_table3)
```

```
Cluster distribution: [905 116]
Cluster distribution: [677 116 228]
Clusters vs Exposure:
Exposure   DENV  naive
cluster_2
0           402    503
1            66     50
Exposure   DENV  naive
cluster_3
0           326    351
1            66     50
2            76    152
```

The clustering groups seem to have odd proportions of DENV and naive patients.

In [13]:

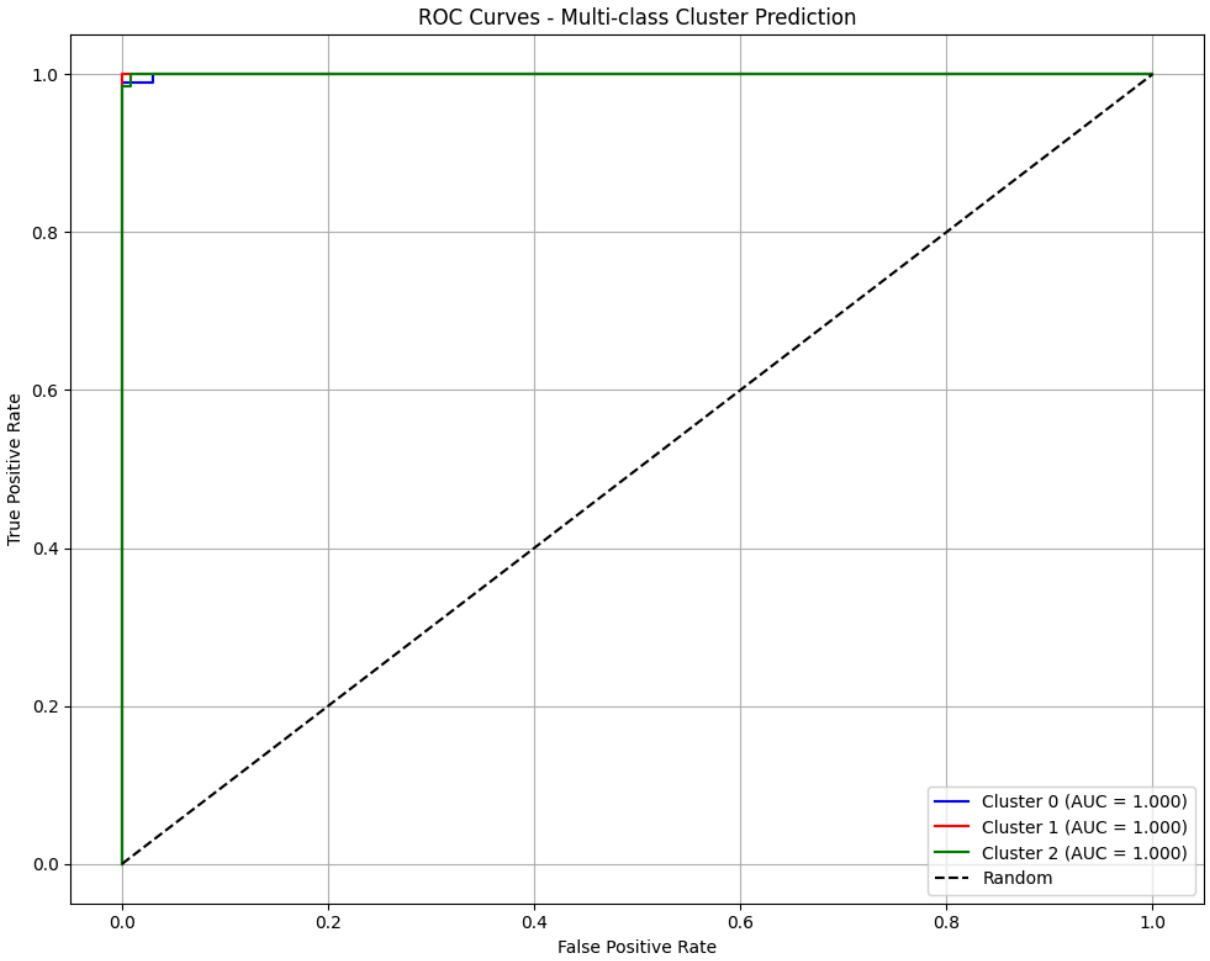
```python
# SVM for cluster prediction
y_clusters_train_3 = metadata_with_clusters['cluster_3'].iloc[train_idx]
y_clusters_test_3 = metadata_with_clusters['cluster_3'].iloc[test_idx]

y_clusters_train_2 = metadata_with_clusters['cluster_2'].iloc[train_idx]
y_clusters_test_2 = metadata_with_clusters['cluster_2'].iloc[test_idx]
# Multi-class SVM
svm_multiclass = Pipeline([
    ('scaler', StandardScaler()),
    ('svm', SVC(kernel='rbf', probability=True, random_state=42))
])
# fit 3 clusters
svm_multiclass.fit(X_train, y_clusters_train_3)
y_clusters_pred_3 = svm_multiclass.predict(X_test)
y_clusters_proba_3 = svm_multiclass.predict_proba(X_test)
# fit 2 clusters
svm_multiclass.fit(X_train, y_clusters_train_2)
y_clusters_pred_2 = svm_multiclass.predict(X_test)
y_clusters_proba_2 = svm_multiclass.predict_proba(X_test)


# Multi-class accuracy
from sklearn.metrics import accuracy_score, confusion_matrix
cluster_accuracy = accuracy_score(y_clusters_test_3, y_clusters_pred_3)
print(f"Cluster Prediction Accuracy (3 clusters): {cluster_accuracy:.3f}")

cluster_accuracy = accuracy_score(y_clusters_test_2, y_clusters_pred_2)
print(f"Cluster Prediction Accuracy (2 clusters): {cluster_accuracy:.3f}")


print("\nConfusion Matrix:")
print(confusion_matrix(y_clusters_test_3, y_clusters_pred_3))
print(confusion_matrix(y_clusters_test_2, y_clusters_pred_2))
```

```
Cluster Prediction Accuracy (3 clusters): 0.987
Cluster Prediction Accuracy (2 clusters): 0.997

Confusion Matrix:
[[204   0   2]
 [  1  31   0]
 [  1   0  68]]
[[275   0]
 [  1  31]]
```

The confusion matrix seems to indicate that fitting our data to 2 clusters is just nearly as good as fitting it to 3. This might indicate that there are more subtle biological relationships among out patients' gene expression than we realized at first. This was evident in assignment 3, when we saw that there were potentially 30 different gene expression patterns among our patients. However, it is more likely that the model could just be fitting to patient immune subtypes/age/sex differences.

In [14]:

```python
# Multi-class ROC (One-vs-Rest)
from sklearn.metrics import roc_curve, auc
from sklearn.preprocessing import label_binarize

# Binarize labels for multi-class ROC
y_test_bin = label_binarize(y_clusters_test_3, classes=list(range(3)))
n_classes = y_test_bin.shape[1]

# Calculate ROC for each class
fpr_multi = dict()
tpr_multi = dict()
roc_auc_multi = dict()

plt.figure(figsize=(10, 8))
colors = ['blue', 'red', 'green']

for i in range(n_classes):
    fpr_multi[i], tpr_multi[i], _ = roc_curve(y_test_bin[:, i], y_clusters_p
    roc_auc_multi[i] = auc(fpr_multi[i], tpr_multi[i])
    plt.plot(fpr_multi[i], tpr_multi[i], color=colors[i],
             label=f'Cluster {i} (AUC = {roc_auc_multi[i]:.3f})')

plt.plot([0, 1], [0, 1], 'k--', label='Random')
plt.xlabel('False Positive Rate')
plt.ylabel('True Positive Rate')
plt.title('ROC Curves - Multi-class Cluster Prediction')
plt.legend()
plt.grid(True)
plt.tight_layout()
plt.savefig('results/multiclass_roc_curves_clust-30.png', dpi=300, bbox_inch
plt.show()
```



Heatmap Visualization

In [15]:
```
# Get top variable genes for heatmap (since RBF kernel doesn't have coef_)
# Use the top 50 most variable genes from our original selection
top_genes_indices = np.arange(50)  # First 50 genes from data_top5000 (alrea
top_gene_names = data_top5000['Symbol'].iloc[top_genes_indices].values

# Create heatmap data
heatmap_data = expr_matrix.iloc[:, top_genes_indices].T

# Z-score normalization (row-wise for genes)
gene_means = heatmap_data.mean(axis=1)
gene_stds = heatmap_data.std(axis=1)
heatmap_data_scaled = heatmap_data.sub(gene_means, axis=0).div(gene_stds, ax

# Handle any NaN values (genes with zero variance)
heatmap_data_scaled = heatmap_data_scaled.fillna(0)

# Create annotation for samples
sample_colors = {'DENV': 'red', 'naive': 'blue'}
exposure_colors = [sample_colors.get(exp, 'gray') for exp in metadata_aligne

cluster_colors = ['gold', 'lightgreen', 'lightcoral']
cluster_sample_colors = [cluster_colors[c] for c in metadata_with_clusters['
```

```
print(f"Heatmap shape: {heatmap_data_scaled.shape}")
print(f"Top {len(top_gene_names)} most variable genes selected")
print(f"Using genes with highest variance from original selection")
```

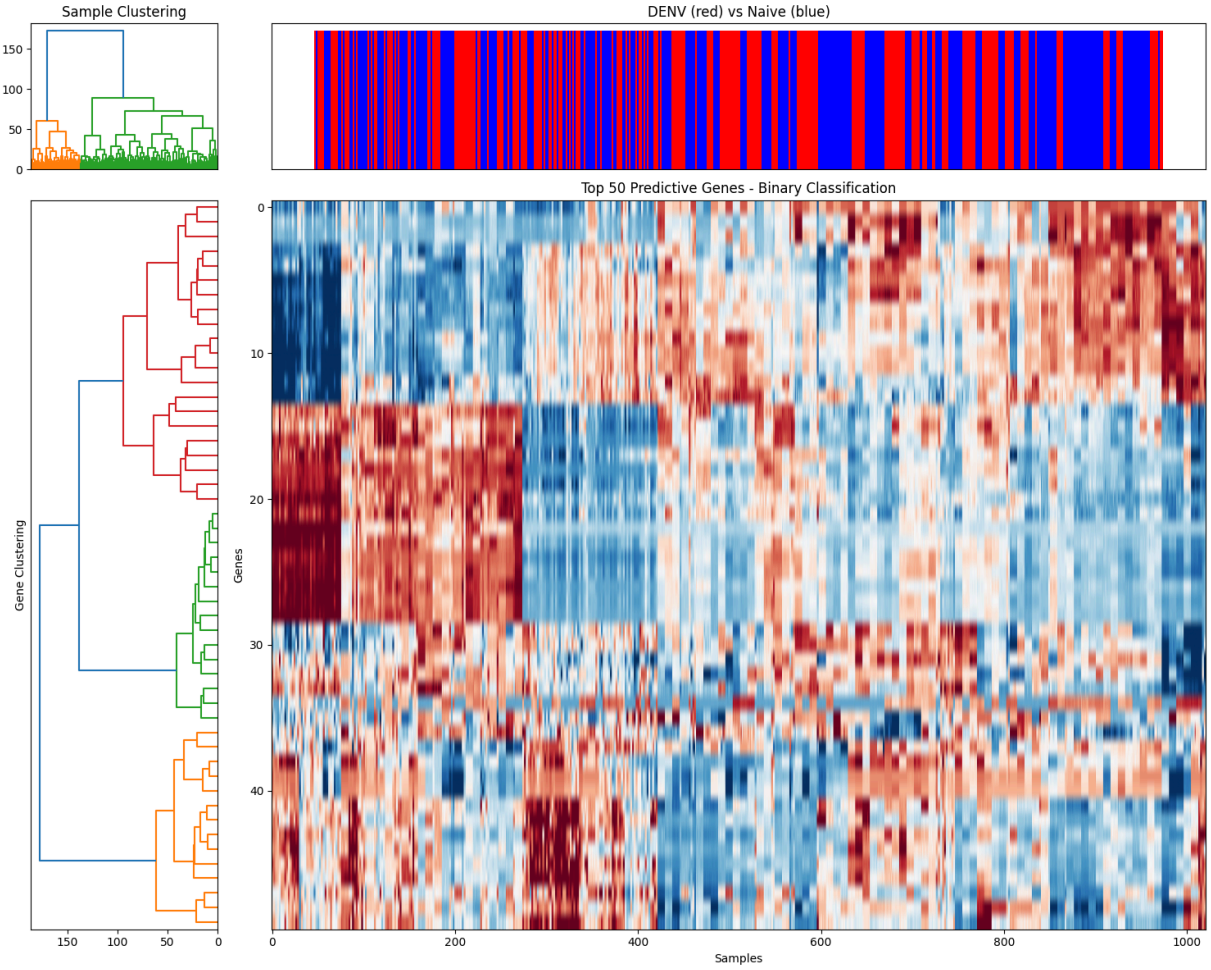
```
Heatmap shape: (50, 1021)
Top 50 most variable genes selected
Using genes with highest variance from original selection
```

In [16]:

```
# Binary Classification Heatmap
from scipy.cluster.hierarchy import dendrogram, linkage
from scipy.spatial.distance import pdist

plt.figure(figsize=(15, 10))

# Calculate linkage for dendrogram
sample_linkage = linkage(pdist(heatmap_data_scaled.T), method='ward')
gene_linkage = linkage(pdist(heatmap_data_scaled), method='ward')

# Create subplots
fig, ((ax_dendro_col, ax_colorbar), (ax_dendro_row, ax_heatmap)) = plt.subpl
    2, 2, figsize=(15, 12),
    gridspec_kw={'height_ratios': [0.2, 1], 'width_ratios': [0.2, 1]}
)

# Column dendrogram
dendro_col = dendrogram(sample_linkage, ax=ax_dendro_col, orientation='top',
ax_dendro_col.set_xticks([])
ax_dendro_col.set_title('Sample Clustering')

# Row dendrogram
dendro_row = dendrogram(gene_linkage, ax=ax_dendro_row, orientation='left',
ax_dendro_row.set_yticks([])
ax_dendro_row.set_ylabel('Gene Clustering')

# Reorder data based on dendrograms
col_order = dendro_col['leaves']
row_order = dendro_row['leaves']

heatmap_ordered = heatmap_data_scaled.iloc[row_order, col_order]

# Main heatmap
im = ax_heatmap.imshow(heatmap_ordered, cmap='RdBu_r', aspect='auto', vmin=-
ax_heatmap.set_title('Top 50 Predictive Genes - Binary Classification')
ax_heatmap.set_xlabel('Samples')
ax_heatmap.set_ylabel('Genes')

# Color bar for exposure
exposure_ordered = [exposure_colors[i] for i in col_order]
ax_colorbar.bar(range(len(exposure_ordered)), [1]*len(exposure_ordered),
               color=exposure_ordered, width=1.0)
ax_colorbar.set_title('DENV (red) vs Naive (blue)')
ax_colorbar.set_xticks([])
ax_colorbar.set_yticks([])

plt.tight_layout()
```

```
plt.savefig('results/binary_heatmap_dendrogram.png', dpi=300, bbox_inches='t
plt.show()
```

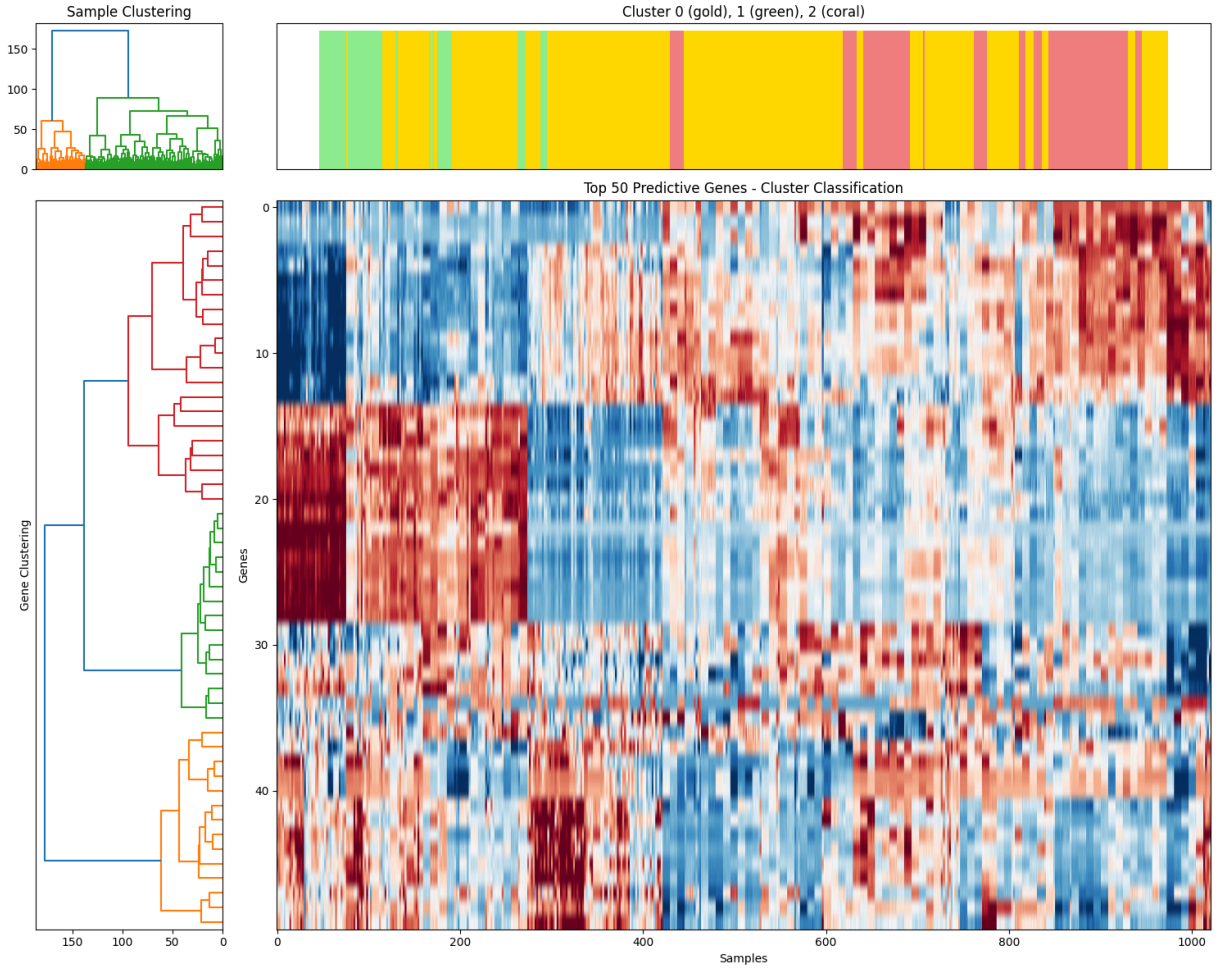
```
<Figure size 1500x1000 with 0 Axes>
```



As expected from what we've learned from assignment 3, we can see that there is indeed a significant relationship between patients prior infected with Dengue virus becoming later infected with zika. However, from the dendrogram, we can the potential beginnings of a third clustering group that might indicate other factors we have not considered.

In [17]:
```
# Cluster Prediction Heatmap
fig, ((ax_dendro_col2, ax_colorbar2), (ax_dendro_row2, ax_heatmap2)) = plt.s
    2, 2, figsize=(15, 12),
    gridspec_kw={'height_ratios': [0.2, 1], 'width_ratios': [0.2, 1]}
)

# Same dendrograms (genes and samples don't change)
dendrogram(sample_linkage, ax=ax_dendro_col2, orientation='top', no_labels=T
ax_dendro_col2.set_xticks([])
ax_dendro_col2.set_title('Sample Clustering')

dendrogram(gene_linkage, ax=ax_dendro_row2, orientation='left', no_labels=Tr
ax_dendro_row2.set_yticks([])
ax_dendro_row2.set_ylabel('Gene Clustering')
```

```
# Main heatmap (same gene ordering)
im2 = ax_heatmap2.imshow(heatmap_ordered, cmap='RdBu_r', aspect='auto', vmir
ax_heatmap2.set_title('Top 50 Predictive Genes - Cluster Classification')
ax_heatmap2.set_xlabel('Samples')
ax_heatmap2.set_ylabel('Genes')

# Color bar for clusters
cluster_ordered = [cluster_sample_colors[i] for i in col_order]
ax_colorbar2.bar(range(len(cluster_ordered)), [1]*len(cluster_ordered),
                 color=cluster_ordered, width=1.0)
ax_colorbar2.set_title('Cluster 0 (gold), 1 (green), 2 (coral)')
ax_colorbar2.set_xticks([])
ax_colorbar2.set_yticks([])

plt.tight_layout()
plt.savefig('results/cluster_heatmap_dendrogram.png', dpi=300, bbox_inches='
plt.show()
```



This clustering heatmap / dendrogram seems to reflect the binary classification that we had done earlier. However, from here we can also see that the cluster bands do not seem to be reflecting any particular groups from binary classification. Further analysis would be necessary, but for now it could be that there are other factors effecting the clustering results more than the DENV vs naive label.

Results Summary

In [18]:

```
# Save final results summary
results_summary = {
    'Binary_Classification_AUC': auc_score,
    'Cluster_Prediction_Accuracy': cluster_accuracy,
    'N_Training_Samples': X_train.shape[0],
    'N_Test_Samples': X_test.shape[0],
    'N_Genes': X_train.shape[1],
    'N_Patients_Train': len(train_patient_ids),
    'N_Patients_Test': len(test_patient_ids)
}

# Save to file
import json
with open('results/svm_results_summary.json', 'w') as f:
    json.dump(results_summary, f, indent=2)

print("=== SVM Analysis Results ===")
print(f"Binary Classification (DENV vs Naive):")
print(f"  - AUC Score: {auc_score:.3f}")
print(f"  - Test Samples: {X_test.shape[0]}")

print(f"\nCluster Prediction:")
print(f"  - Accuracy: {cluster_accuracy:.3f}")
print(f"  - Number of Clusters: 3")

print(f"\nData Split:")
print(f"  - Training: {X_train.shape[0]} samples from {len(train_patient_ids
print(f"  - Testing: {X_test.shape[0]} samples from {len(test_patient_ids)}
print(f"  - Features: {X_train.shape[1]} genes")

print(f"\nFiles saved to results/:")
print("  - binary_roc_curve.png")
print("  - multiclass_roc_curves.png")
print("  - binary_heatmap_dendrogram.png")
print("  - cluster_heatmap_dendrogram.png")
print("  - svm_results_summary.json")
```

```
=== SVM Analysis Results ===
Binary Classification (DENV vs Naive):
  - AUC Score: 0.968
  - Test Samples: 307

Cluster Prediction:
  - Accuracy: 0.997
  - Number of Clusters: 3

Data Split:
  - Training: 714 samples from 714 patients
  - Testing: 307 samples from 307 patients
  - Features: 4847 genes

Files saved to results/:
  - binary_roc_curve.png
  - multiclass_roc_curves.png
  - binary_heatmap_dendrogram.png
  - cluster_heatmap_dendrogram.png
  - svm_results_summary.json
```

In [19]:

```python
# 1. Binary Classification Predictions
binary_predictions = pd.DataFrame({
    'sample_id': X_test.index,
    'true_label': [label_encoder.classes_[i] for i in y_test_binary],
    'predicted_label': [label_encoder.classes_[i] for i in y_pred],
    'predicted_correctly': y_test_binary == y_pred,
    'probability_class_0': svm_binary.predict_proba(X_test)[:, 0],
    'probability_class_1': svm_binary.predict_proba(X_test)[:, 1],
    'confidence': np.max(svm_binary.predict_proba(X_test), axis=1)
})

# Add patient information
binary_predictions['patient_id'] = metadata_aligned.loc[binary_predictions['

print("=== Binary Classification Predictions ===")
print(f"Total test samples: {len(binary_predictions)}")
print(f"Correctly predicted: {binary_predictions['predicted_correctly'].sum(
print(f"Accuracy: {binary_predictions['predicted_correctly'].mean():.3f}")
print("\nFirst 10 predictions:")
display(binary_predictions.head(10))

# 2. Cluster Classification Predictions (3 clusters)
cluster_predictions_3 = pd.DataFrame({
    'sample_id': X_test.index,
    'true_cluster': y_clusters_test_3.values,
    'predicted_cluster': y_clusters_pred_3,
    'predicted_correctly': y_clusters_test_3.values == y_clusters_pred_3,
})

# Add probabilities for each cluster
for i in range(3):
    cluster_predictions_3[f'probability_cluster_{i}'] = y_clusters_proba_3[:

cluster_predictions_3['confidence'] = np.max(y_clusters_proba_3, axis=1)
cluster_predictions_3['patient_id'] = metadata_aligned.loc[cluster_predictic
```

```
cluster_predictions_3['exposure'] = metadata_aligned.loc[cluster_predictions

print("\n=== 3-Cluster Classification Predictions ===")
print(f"Total test samples: {len(cluster_predictions_3)}")
print(f"Correctly predicted: {cluster_predictions_3['predicted_correctly'].s
print(f"Accuracy: {cluster_predictions_3['predicted_correctly'].mean():.3f}"
print("\nFirst 10 predictions:")
display(cluster_predictions_3.head(10))

# 3. Cluster Classification Predictions (2 clusters)
cluster_predictions_2 = pd.DataFrame({
    'sample_id': X_test.index,
    'true_cluster': y_clusters_test_2.values,
    'predicted_cluster': y_clusters_pred_2,
    'predicted_correctly': y_clusters_test_2.values == y_clusters_pred_2,
})

# Add probabilities for each cluster
for i in range(2):
    cluster_predictions_2[f'probability_cluster_{i}'] = y_clusters_proba_2[:

cluster_predictions_2['confidence'] = np.max(y_clusters_proba_2, axis=1)
cluster_predictions_2['patient_id'] = metadata_aligned.loc[cluster_predictic
cluster_predictions_2['exposure'] = metadata_aligned.loc[cluster_predictions

print("\n=== 2-Cluster Classification Predictions ===")
print(f"Total test samples: {len(cluster_predictions_2)}")
print(f"Correctly predicted: {cluster_predictions_2['predicted_correctly'].s
print(f"Accuracy: {cluster_predictions_2['predicted_correctly'].mean():.3f}"
print("\nFirst 10 predictions:")
display(cluster_predictions_2.head(10))

# 4. Summary by Patient (for binary classification)
patient_summary = binary_predictions.groupby('patient_id').agg({
    'predicted_correctly': ['count', 'sum', 'mean'],
    'confidence': 'mean',
    'true_label': 'first'
}).round(3)

patient_summary.columns = ['total_samples', 'correct_predictions', 'accuracy
patient_summary = patient_summary.reset_index()

print("\n=== Patient-Level Summary (Binary Classification) ===")
print("Accuracy per patient:")
display(patient_summary.head(10))

# 5. Misclassified Samples Analysis
misclassified_binary = binary_predictions[~binary_predictions['predicted_cor
misclassified_clusters_3 = cluster_predictions_3[~cluster_predictions_3['pre

print(f"\n=== Misclassified Samples ===")
print(f"Binary classification errors: {len(misclassified_binary)}")
if len(misclassified_binary) > 0:
    print("Binary misclassifications:")
    display(misclassified_binary[['sample_id', 'patient_id', 'true_label', '
```

```
print(f"\n3-cluster classification errors: {len(misclassified_clusters_3)}")
if len(misclassified_clusters_3) > 0:
    print("Cluster misclassifications:")
    display(misclassified_clusters_3[['sample_id', 'patient_id', 'true_clust

# 6. Save all prediction matrices
os.makedirs('results', exist_ok=True)
binary_predictions.to_csv('results/svm_binary_predictions.csv', index=False)
cluster_predictions_3.to_csv('results/svm_cluster3_predictions.csv', index=F
cluster_predictions_2.to_csv('results/svm_cluster2_predictions.csv', index=F
patient_summary.to_csv('results/svm_patient_summary.csv', index=False)

print(f"\n=== Files Saved ===")
print("- results/svm_binary_predictions.csv")
print("- results/svm_cluster3_predictions.csv")
print("- results/svm_cluster2_predictions.csv")
print("- results/svm_patient_summary.csv")
```

```
=== Binary Classification Predictions ===
Total test samples: 307
Correctly predicted: 276
Accuracy: 0.899

First 10 predictions:
```

	sample_id	true_label	predicted_label	predicted_correctly	probability_class_0	pr
0	SRR8908211	DENV	DENV	True	0.870046	
1	SRR8908804	naive	naive	True	0.004230	
2	SRR8907955	DENV	DENV	True	0.980159	
3	SRR8907963	DENV	DENV	True	0.914416	
4	SRR8908590	DENV	DENV	True	0.977113	
5	SRR8908395	DENV	DENV	True	0.990573	
6	SRR8908772	naive	DENV	False	0.649143	
7	SRR8908745	DENV	DENV	True	0.984448	
8	SRR8908477	naive	naive	True	0.039715	
9	SRR8908303	DENV	naive	False	0.421954	

```
=== 3-Cluster Classification Predictions ===
Total test samples: 307
Correctly predicted: 303
Accuracy: 0.987

First 10 predictions:
```

	sample_id	true_cluster	predicted_cluster	predicted_correctly	probability_cluster_
0	SRR8908211	2	2	True	0.0055
1	SRR8908804	2	2	True	0.0001
2	SRR8907955	0	0	True	0.9975
3	SRR8907963	0	0	True	0.9862
4	SRR8908590	0	0	True	0.9758
5	SRR8908395	0	0	True	0.9987
6	SRR8908772	0	0	True	0.9999
7	SRR8908745	1	1	True	0.0110
8	SRR8908477	0	0	True	0.9805
9	SRR8908303	1	1	True	0.0000

```
=== 2-Cluster Classification Predictions ===
Total test samples: 307
Correctly predicted: 306
Accuracy: 0.997

First 10 predictions:
```

	sample_id	true_cluster	predicted_cluster	predicted_correctly	probability_cluster_
0	SRR8908211	0	0	True	0.9991
1	SRR8908804	0	0	True	0.9975
2	SRR8907955	0	0	True	0.9990
3	SRR8907963	0	0	True	0.9885
4	SRR8908590	0	0	True	0.9893
5	SRR8908395	0	0	True	0.9990
6	SRR8908772	0	0	True	0.9999
7	SRR8908745	1	1	True	0.0074
8	SRR8908477	0	0	True	0.9998
9	SRR8908303	1	1	True	0.0000

```
=== Patient-Level Summary (Binary Classification) ===
Accuracy per patient:
```

	patient_id	total_samples	correct_predictions	accuracy	avg_confidence	true_exp
0	SRR8907881	1	1	1.0	0.984	
1	SRR8907882	1	1	1.0	0.967	
2	SRR8907884	1	1	1.0	0.952	
3	SRR8907888	1	1	1.0	0.699	
4	SRR8907889	1	1	1.0	0.990	
5	SRR8907902	1	1	1.0	0.909	
6	SRR8907904	1	1	1.0	0.930	
7	SRR8907908	1	1	1.0	0.922	
8	SRR8907909	1	1	1.0	0.937	
9	SRR8907910	1	1	1.0	0.940	

```
=== Misclassified Samples ===
Binary classification errors: 31
Binary misclassifications:
```

	sample_id	patient_id	true_label	predicted_label	confidence
6	SRR8908772	SRR8908772	naive	DENV	0.649143
9	SRR8908303	SRR8908303	DENV	naive	0.578046
18	SRR8908515	SRR8908515	naive	DENV	0.515042
21	SRR8908145	SRR8908145	naive	DENV	0.940442
33	SRR8908302	SRR8908302	DENV	naive	0.528094
37	SRR8908144	SRR8908144	naive	DENV	0.958716
41	SRR8908427	SRR8908427	naive	DENV	0.969407
67	SRR8908472	SRR8908472	naive	DENV	0.602684
98	SRR8908835	SRR8908835	naive	DENV	0.751930
105	SRR8908426	SRR8908426	naive	DENV	0.961974
119	SRR8908413	SRR8908413	naive	DENV	0.931354
120	SRR8908446	SRR8908446	DENV	naive	0.584913
128	SRR8908437	SRR8908437	DENV	naive	0.818729
143	SRR8908536	SRR8908536	naive	DENV	0.711836
147	SRR8907942	SRR8907942	naive	DENV	0.588806
162	SRR8907946	SRR8907946	naive	DENV	0.600409
163	SRR8907945	SRR8907945	naive	DENV	0.505188
177	SRR8908573	SRR8908573	DENV	naive	0.815127
193	SRR8908231	SRR8908231	DENV	naive	0.902214
198	SRR8908193	SRR8908193	naive	DENV	0.571512
199	SRR8908412	SRR8908412	naive	DENV	0.954664
209	SRR8907944	SRR8907944	naive	DENV	0.578198
240	SRR8908447	SRR8908447	DENV	naive	0.551624
243	SRR8908574	SRR8908574	DENV	naive	0.947419
253	SRR8908821	SRR8908821	naive	DENV	0.605829
257	SRR8908378	SRR8908378	DENV	naive	0.766264
276	SRR8908572	SRR8908572	DENV	naive	0.812559
288	SRR8908230	SRR8908230	DENV	naive	0.911921
294	SRR8908895	SRR8908895	DENV	naive	0.575062
296	SRR8907975	SRR8907975	naive	DENV	0.634732
298	SRR8907939	SRR8907939	naive	DENV	0.564950

```
3-cluster classification errors: 4
Cluster misclassifications:
```

	sample_id	patient_id	true_cluster	predicted_cluster	confidence	exposure
27	SRR8907949	SRR8907949	0	2	0.677799	naive
36	SRR8907951	SRR8907951	0	2	0.650708	naive
145	SRR8908616	SRR8908616	2	0	0.593017	naive
160	SRR8908135	SRR8908135	1	0	0.841984	DENV

```
=== Files Saved ===
- results/svm_binary_predictions.csv
- results/svm_cluster3_predictions.csv
- results/svm_cluster2_predictions.csv
- results/svm_patient_summary.csv
```

In [20]:
```python
# Check how well clusters align with exposure
from sklearn.metrics import adjusted_rand_score

# Convert exposure to binary for comparison
exposure_binary = label_encoder.fit_transform(metadata_aligned['Exposure'])
cluster_labels_2_all = metadata_with_clusters['cluster_2']

# Calculate agreement between clusters and exposure
ari_score = adjusted_rand_score(exposure_binary, cluster_labels_2_all)
print(f"Adjusted Rand Index between clusters and exposure: {ari_score:.3f}")

# Show the crosstab more clearly
print("\nCluster vs Exposure crosstab:")
crosstab = pd.crosstab(cluster_labels_2_all, metadata_aligned['Exposure'], m
print(crosstab)

# Calculate what percentage of each cluster is each exposure
crosstab_pct = pd.crosstab(cluster_labels_2_all, metadata_aligned['Exposure'
print("\nPercentage breakdown:")
print(crosstab_pct)
```

```
Adjusted Rand Index between clusters and exposure: 0.009

Cluster vs Exposure crosstab:
Exposure   DENV  naive   All
cluster_2
0           402    503   905
1            66     50   116
All         468    553  1021

Percentage breakdown:
Exposure       DENV     naive
cluster_2
0          0.444199  0.555801
1          0.568966  0.431034
```